

计算思维 R语言简介

2025-2026秋季学期



R语言简介

R语言是一种自由软件编程语言与操作环境，主要用于统计分析、绘图以及数据挖掘。R由新西兰奥克兰大学的统计学家罗斯·伊哈卡和罗伯特·杰特曼开发，现在由R核心小组负责开发，同时也有其他用户编写了诸多外挂的软件包。

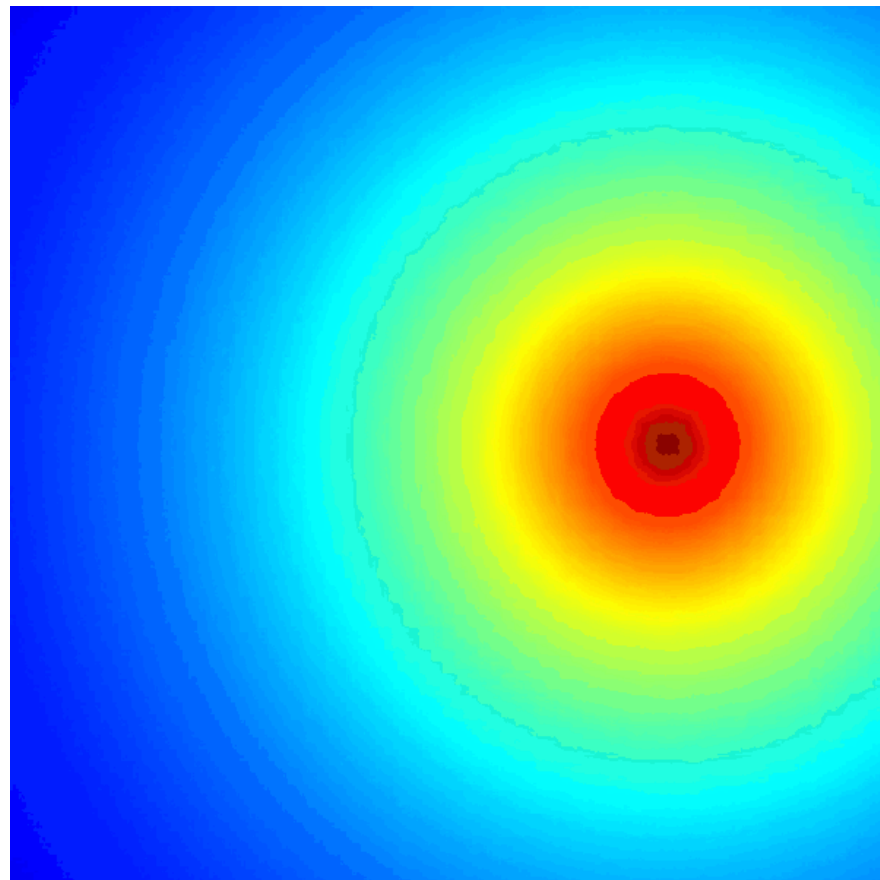
R语言简介

R主要用于数据分析。用户可以用R来进行一些基本的统计检验，构建线性及非线性的模型，对时间序列加以分析，或对数据进行分类与聚类分析。R的另一强项是绘图功能，画出的图表能够达到专业出版物的要求，也可加入数学符号。计算强度较大时，用户可在程序中嵌入C、C++以及FORTRAN语言以帮助运算。

R语言简介



R语言LOGO



R语言用14行代码创建的图形

为什么要学R语言

1、广泛应用：

R语言在生物信息学领域被广泛应用，许多生物信息学家和生物学家都使用R语言进行数据分析和可视化。

2、强大的数据处理和统计功能：

R语言提供了丰富的数据处理和统计分析功能，包括数据清洗、转换、统计测试、回归分析等，非常适合处理生物学实验数据。

3、丰富的可视化功能：

R语言拥有众多用于数据可视化的包和函数，可以创建各种类型的图表和图形，帮助研究人员更好地理解 and 解释数据。

为什么要学R语言

4、开源免费：

R语言是开源的，免费使用，而且有一个庞大的社区支持，可以轻松获取各种教程、文档和资源。

5、与生物信息学相关的包和工具：

R语言有许多与生物信息学相关的包和工具，例如 [Bioconductor](#)，提供了丰富的生物信息学分析工具和数据集。



怎么学习R语言

编程语言是人与计算机沟通的一种形式语言，根据设计好的编程元素和语法规则，来严格规范地表达我们想要做的事情的每一步（程序代码），使得计算机能够明白并正确执行，得到期望的结果。

- 错误的编程：“学编程，就是照着别人的代码敲代码”——事倍功半，关键是学不会真正的编程！
- 正确的编程：先学习并掌握编程元素和语法规则，比如数据结构（容器）、分支/循环结构、自定义函数等，然后遇到具体问题，分解问题、借助实例梳理，根据掌握的编程元素和语法规则翻译成代码并调试通过，从而自己写出代码解决问题。

怎么学习R语言

编程语言是人与计算机沟通的一种形式语言，根据设计好的编程元素和语法规则，来严格规范地表达我们想要做的事情的每一步（程序代码），使得计算机能够明白并正确执行，得到期望的结果。

- 错误的编程：“学编程，就是照着别人的代码敲代码”——事倍功半，关键是学不会真正的编程！
- 正确的编程：先学习并掌握编程元素和语法规则，比如数据结构（容器）、分支/循环结构、自定义函数等，然后遇到具体问题，分解问题、借助实例梳理，根据掌握的编程元素和语法规则翻译成代码并调试通过，从而自己写出代码解决问题。

怎么学习R语言

R语言学习资源——书籍：

- "R语言实战" (R for Data Science) : 由Hadley Wickham和Garrett Grolemund合著, 是学习R语言和数据科学的经典教材。
- "R语言数据分析" (An Introduction to Statistical Learning with Applications in R) : James Witten、Hastie和Tibshirani合著, 介绍了统计学习方法在R语言中的应用。
- "R语言编程艺术" (The Art of R Programming) : 由Norman Matloff撰写, 介绍了R语言的编程技巧和最佳实践。

怎么学习R语言

R语言学习资源——在线课程：

- Coursera上的课程：例如Johns Hopkins大学的"R语言数据科学"（Data Science with R）课程，以及其他许多与R语言相关的课程。
- edX上的课程：例如Harvard大学的"R语言基础"（Introduction to R for Data Science）课程。
- DataCamp：专注于数据科学和数据分析的在线学习平台，提供了许多与R语言相关的课程和实践项目。

怎么学习R语言

R语言学习资源——在线教程和博客：

- R官方网站提供了丰富的教程和文档，适合初学者入门：[R官方网站](#)
- RStudio网站上也有许多与R语言和数据科学相关的教程和资源：RStudio教程
- 数据科学家Hadley Wickham的网站提供了许多有关R语言和数据科学的教程和资源：[Hadley Wickham网站](#)

怎么学习R语言

R语言学习资源——社区论坛和问答网站：

- Stack Overflow上有专门针对R语言的问答板块，可以在这里提出问题并得到解答：[Stack Overflow - R](https://stackoverflow.com/questions/tagged/r)
- RStudio社区论坛是一个讨论R语言和数据科学的活跃社区，可以在这里寻求帮助和交流经验：RStudio Community

R语言学习资源——GitHub上的项目和资源：

- GitHub上有许多与R语言相关的开源项目和资源，可以搜索并浏览这些项目，了解其他人是如何使用R语言进行数据分析和可视化的。

怎么学习R语言

R语言学习资源——社区论坛和问答网站：

- Stack Overflow上有专门针对R语言的问答板块，可以在这里提出问题并得到解答：[Stack Overflow - R](#)
- RStudio社区论坛是一个讨论R语言和数据科学的活跃社区，可以在这里寻求帮助和交流经验：RStudio Community

R语言学习资源——GitHub上的项目和资源：

- GitHub上有许多与R语言相关的开源项目和资源，可以搜索并浏览这些项目，了解其他人是如何使用R语言进行数据分析和可视化的。

R语言编程思想

面向对象的编程

面向**对象**(Object Oriented)是**软件开发方法**，一种编程范式。面向**对象**的概念和应用已超越了**程序设计和软件开发**，扩展到如**数据库系统**、**交互式界面**、应用结构、应用平台、**分布式系统**、**网络管理结构**、**CAD技术**、**人工智能**等领域。面向对象是一种对现实世界理解和**抽象**的方法，是计算机编程技术发展到现在一定阶段后的产物。

面向对象是相对于**面向过程**来讲的，**面向对象方法**，把相关的数据和方法组织为一个整体来看待，从更高的层次来进行系统**建模**，更贴近事物的自然运行模式。^[1]



听听，你说的是人话吗



R语言编程思想

面向过程

(Procedure Oriented 简称PO：如C语言)：

当解决一个问题的时候，面向过程会把事情拆分成：一个个函数和数据（用于方法的参数）。然后按照一定的顺序，执行完这些方法（每个方法看作一个过程），等方法执行完了，事情就搞定了。

面向对象

(Object Oriented简称OO：如C++，JAVA等语言)：

当解决一个问题的时候，面向对象会把事物抽象成对象的概念，就是说这个问题里面有哪些对象，然后给对象赋一些属性和方法，然后让每个对象去执行自己的方法，问题得到解决。

R语言编程思想

假设你是一家披萨店的老板，你需要让系统（程序）来处理顾客点披萨的订单。

➤ 面向过程：

当顾客点了一个“夏威夷披萨”时，你会这样一步步操作：

1. 准备面团：拿出面团，揉一揉，擀成圆形。
2. 准备酱料：在面团上均匀涂抹番茄酱。
3. 准备配料：切好火腿、切好菠萝。
4. 撒上配料：把火腿、菠萝、芝士等撒在酱料上。
5. 烘烤：把披萨放进烤箱，设定200度，烤15分钟。
6. 检查：时间到，拿出来看看是不是金黄酥脆。
7. 装盘：把烤好的披萨切成6块，放到盘子里。
8. 上菜：端给顾客。

R语言编程思想

假设你是一家披萨店的老板，你需要让系统（程序）来处理顾客点披萨的订单。

➤ **面向对象：**

雇佣了一群“披萨师傅”，每个“披萨师傅”都精通一种披萨：

有一个专门做“夏威夷披萨”的师傅。

有一个专门做“海鲜披萨”的师傅。

有一个专门做“玛格丽特披萨”的师傅。



R语言编程思想

假设你是一家披萨店的老板，你需要让系统（程序）来处理顾客点披萨的订单。

➤ **面向对象：**

雇佣了一群“披萨师傅”，每个“披萨师傅”都精通一种披萨：
每个师傅都有：

自己的工具和配方（属性/数据）：

夏威夷师傅知道：“我的配料是火腿和菠萝。”

海鲜师傅知道：“我的配料是虾仁和鱿鱼。”

自己的手艺（方法/函数）：

每个师傅都会 准备面团()、涂抹酱料()、撒配料()、烘烤()、装盘() 这些本领。

R语言编程思想

假设你是一家披萨店的老板，你需要让系统（程序）来处理顾客点披萨的订单。

➤ **面向对象：**

雇佣了一群“披萨师傅”，每个“披萨师傅”都精通一种披萨：
当顾客点了一个“夏威夷披萨”时

1. 你创建一个名为“夏威夷披萨师傅”的对象（或者叫他过来）。
2. 你对他说：“做披萨！”（调用 `make_pizza()` 方法）
3. 这个师傅就会自动按照他脑子里的配方和手艺，完成从准备到上菜的所有步骤。
4. 做完后，他把披萨交给你，你再给顾客。

R语言编程思想

特点	面向过程	面向对象
核心关注点	做什么 (What to do) - 步骤、流程、函数	谁来做 (Who does it) - 对象、数据、行为
组织方式	像一个大厨记着所有菜的菜谱，按步骤执行。	像一个经理管理一群专家员工，每个员工负责自己的领域。
数据与功能	数据和函数是分开的。函数操作外部的数据。	数据（属性）和函数（方法） 捆绑 在一个对象内部。
扩展性	加一种新披萨，要写一整套新步骤，容易重复和出错。	加一种新披萨，只需 创建一个新的“师傅”（类） ，定义好他的配方和手艺，就能用了。
管理复杂性	流程越复杂，代码越长越乱，像一团乱麻。	复杂系统被分解成多个独立的对象，每个对象各司其职，代码更清晰、易维护。

【8分钟搞懂面向对象编程 | 面向过程vs面向对象 | OOP | 封装 继承 多态】
https://www.bilibili.com/video/BV1wZ4y1B7A7/?share_source=copy_web&vd_source=f29334cd47f8c16971c64516ce205b6e

计算机科学与技术学院

R和RStudio

R提供了语言环境，它提供了R的语法规则，编译，扩展包等信息。如果只用它自带的软件包来编写和调试R脚本的话，既耗时又费力，甚至不具有开发完整项目的条件。所以，Rstudio提供了一个人性化的可视界面来操作R语言。在软件里可以创建完整的项目，编写脚本，查看变量值，获取绘图的结果等等。

R和RStudio

R的下载地址：

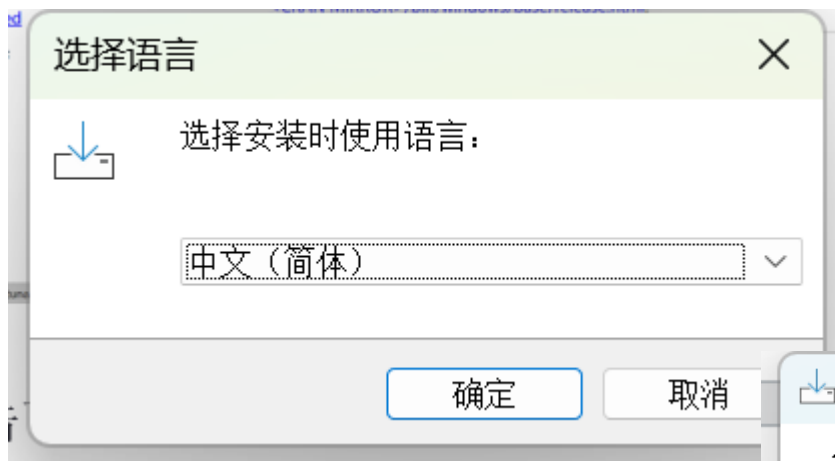
<https://cran.r-project.org/mirrors.html>

RStudio的下载地址：

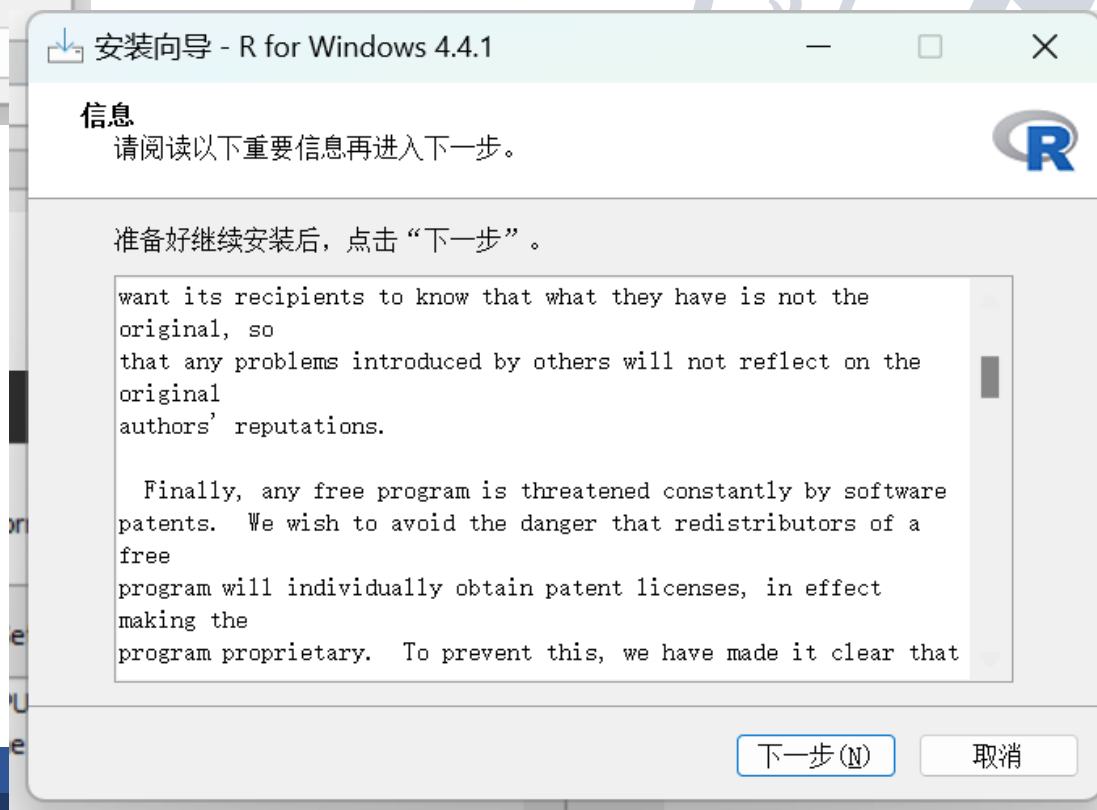
<https://posit.co/download/rstudio-desktop/>

课程群文件中已上传安装包，也可以直接下载使用

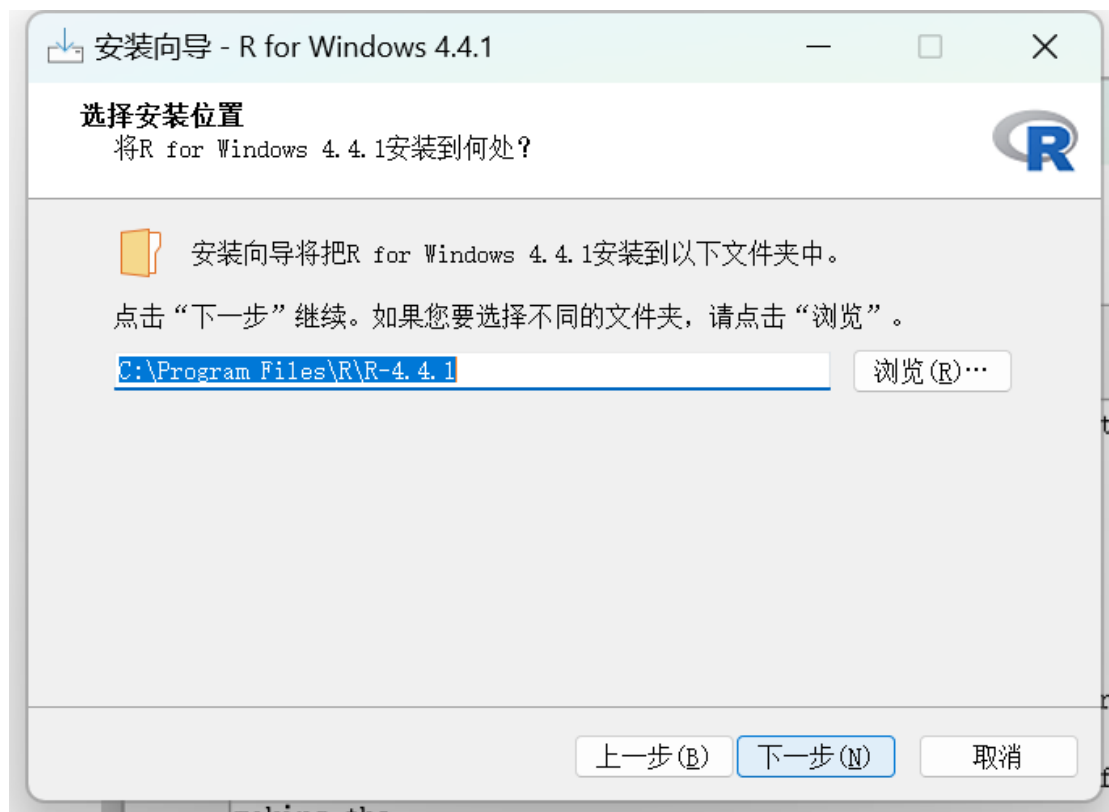
R的安装



1. 选择中文
2. 进入下一步



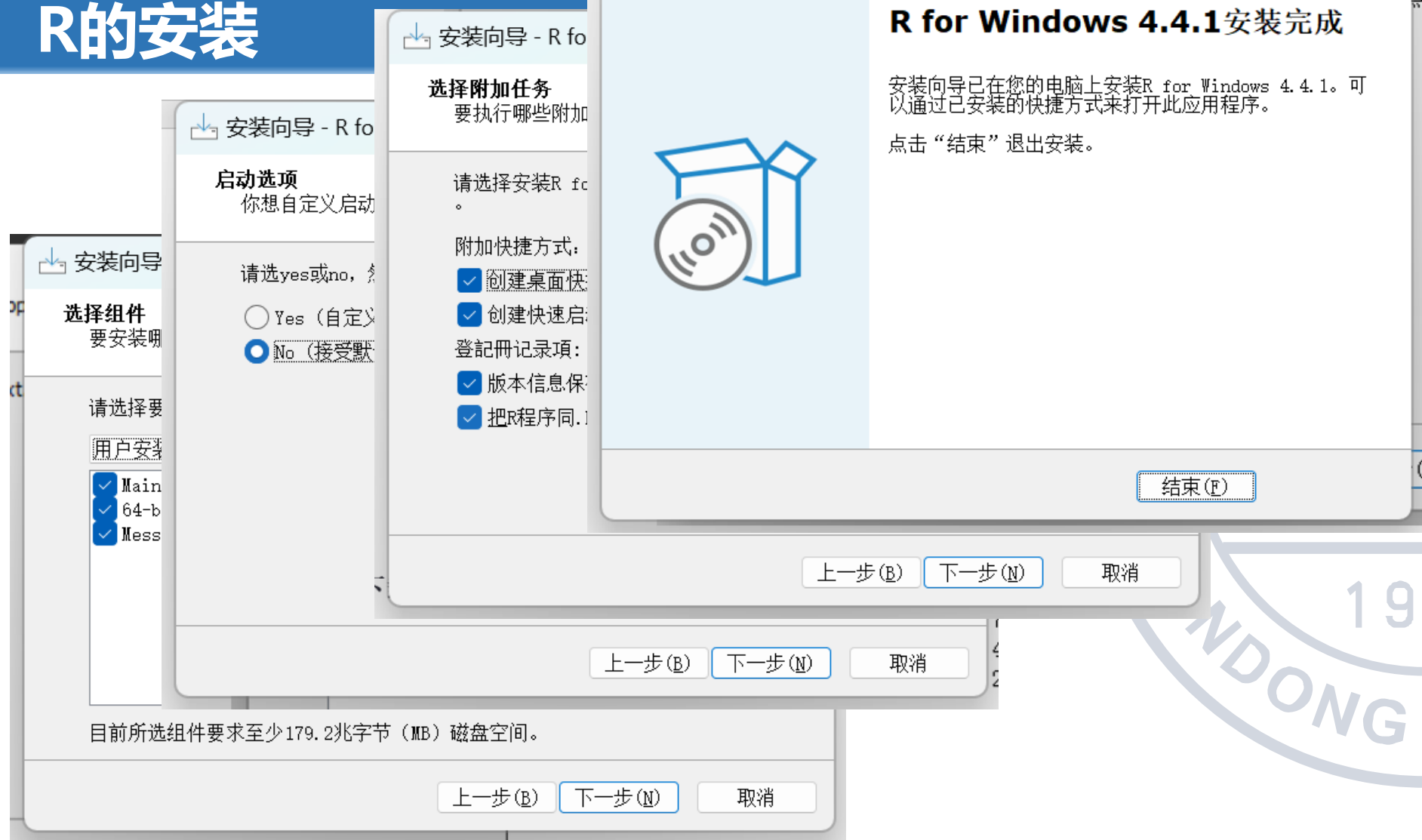
R的安装



3. 选择安装路径，请选择不含中文字符的路径。

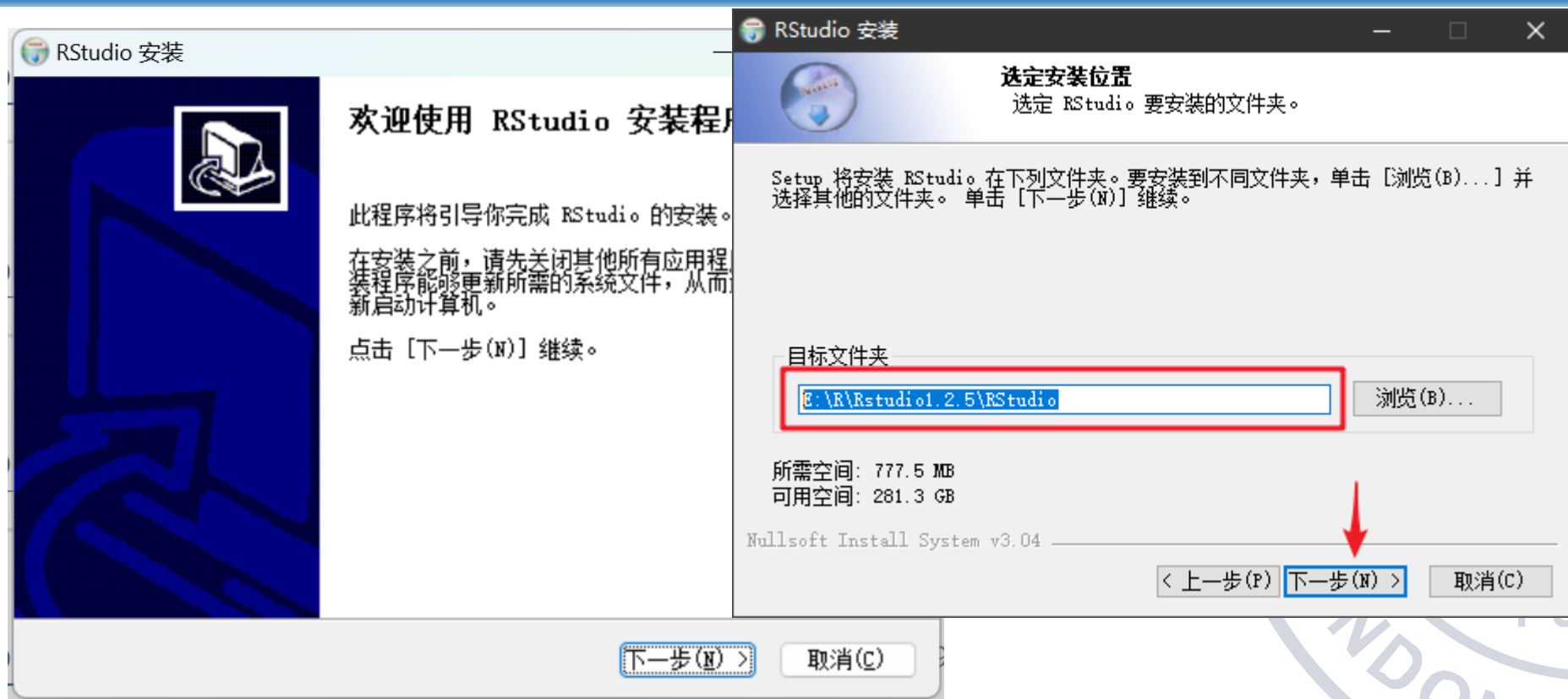


R的安装



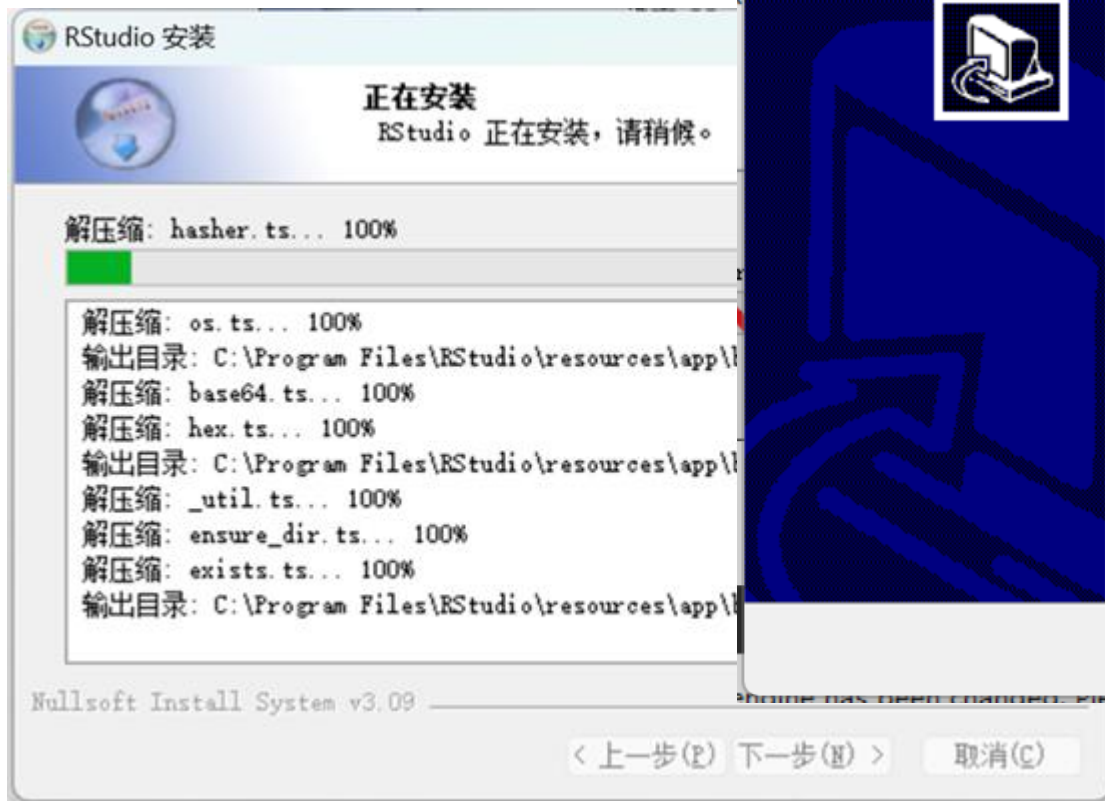
4. 一直点下一步，直到看到安装完成的界面点结束。

RStudio的安装



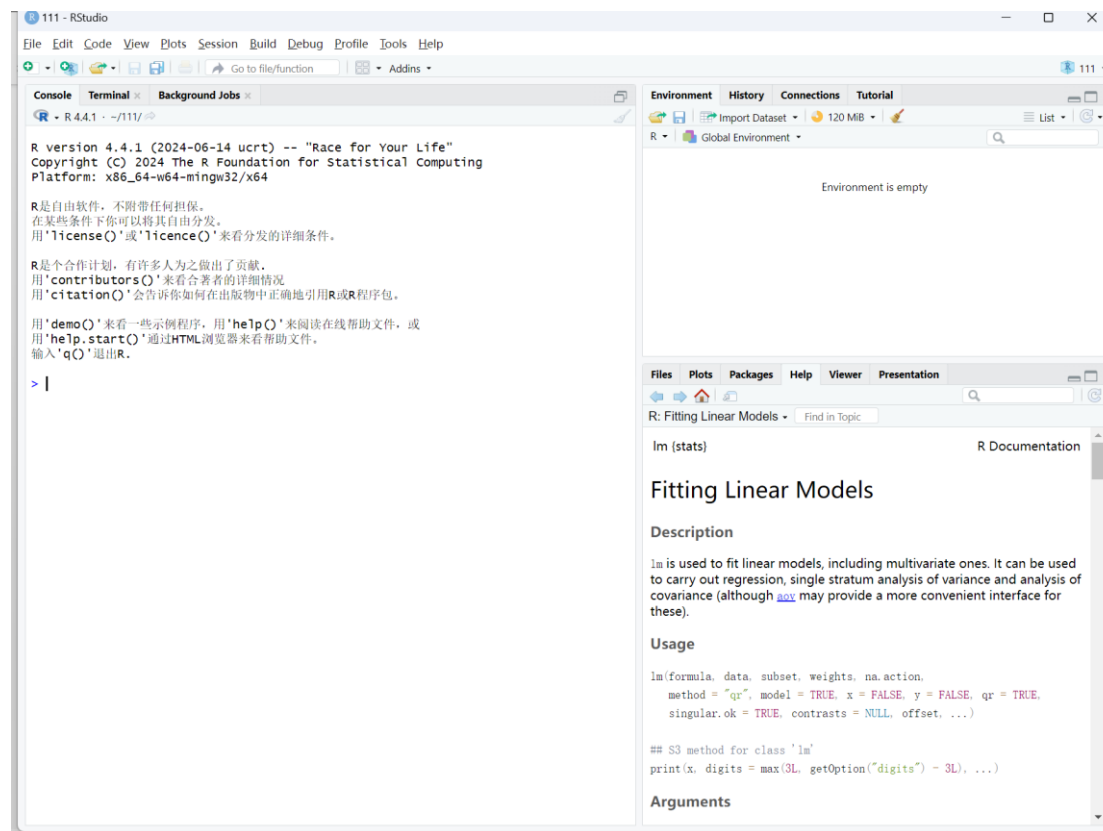
1. 打开安装程序点击下一步。
2. 选择安装位置, 请选择不含中文字符的路径。

RStudio的安装



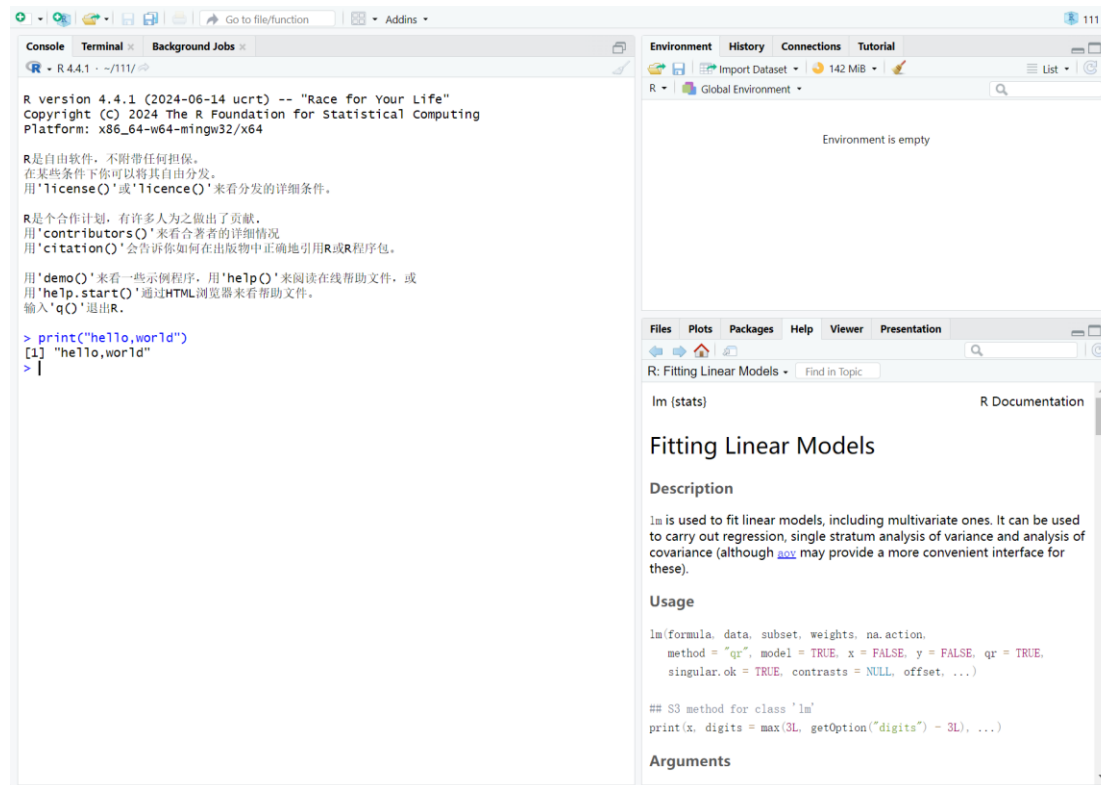
3. 点击下一步安装，等待出现成功安装的界面，点击完成。

RStudio的安装



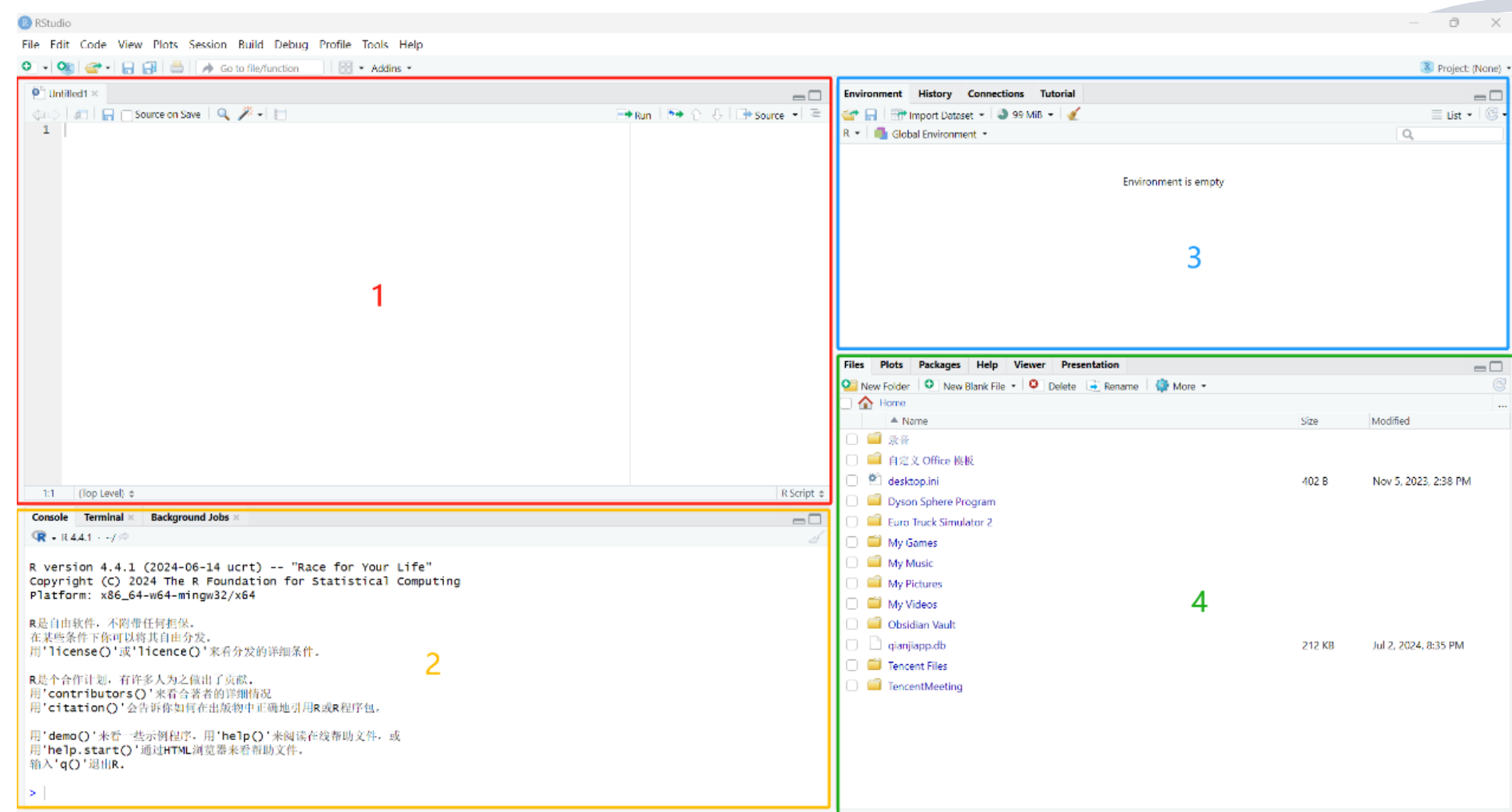
双击打开桌面上安装好的RStudio图标启动程序可以看到上图所示的界面

RStudio的安装



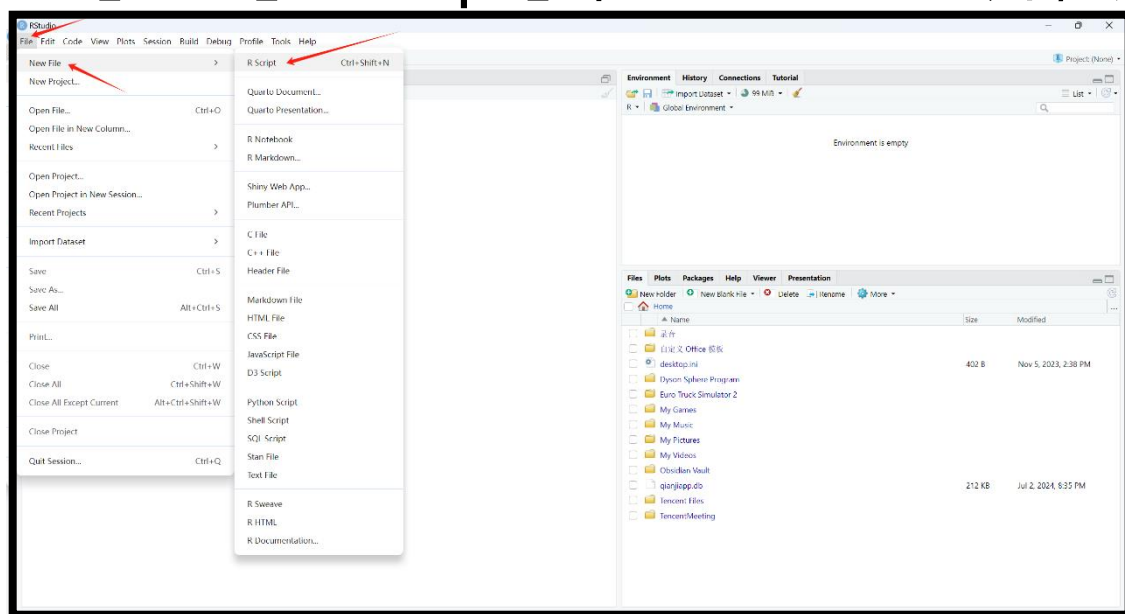
在光标后输入代码：`print("hello,world")`
按ENTER键，观察是否有正确输出`[1] "hello,world"`

RStudio界面介绍

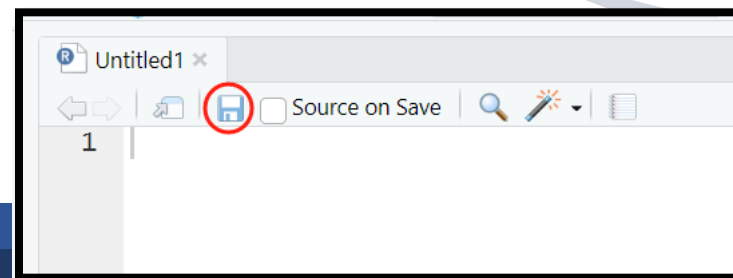


RStudio界面介绍

- Source: (左上角1区) 这个区域主要负责代码的撰写。刚启动时, 可能会看不到这个区。可点击左上角的【File】→【New File】→【R Script】将Source区打开, 如下图。

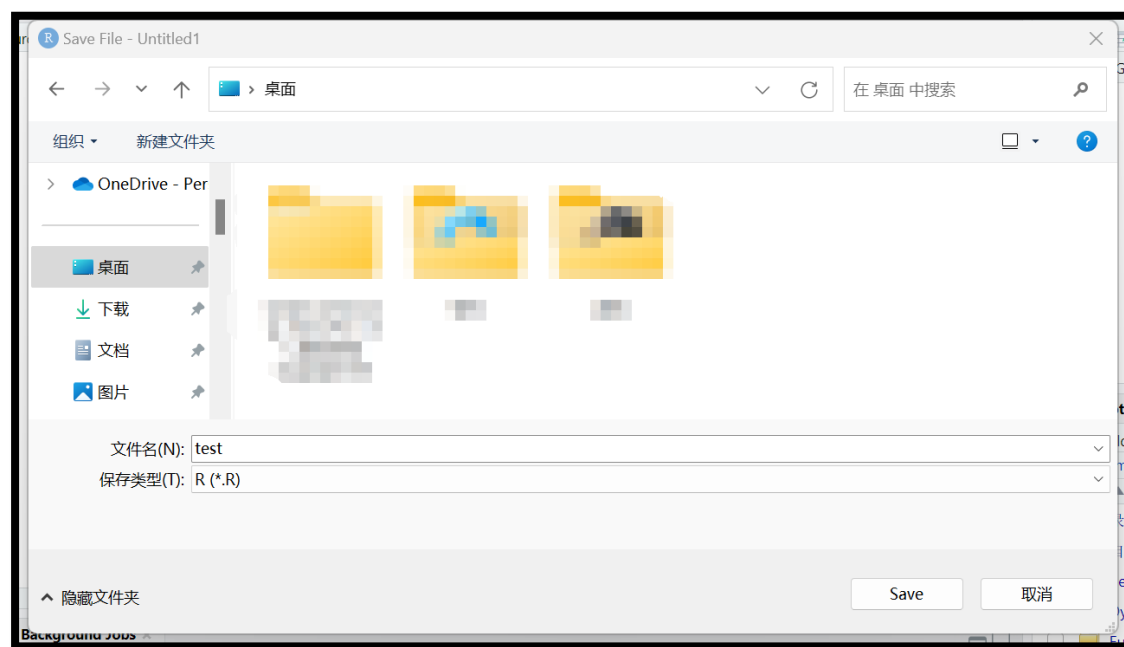


上面步骤将新建一个名为“Untitled1”的R代码文件(后缀名默认为.R)。你可在文件内撰写代码, 然后可按快捷键Ctrl + S(或点击下图红圈里面的按钮)保存文件。



RStudio界面介绍

- Source: (左上角1区)
这个区域主要负责代码的撰写。刚启动时,可能会看不到这个区。可点击左上角的【File】→【New File】→【R Script】将Source区打开,如下图。



接着会跳出“Save File”弹出框,然后将“Untitled1”文件重命名以及选择保存路径。这里重命名为“test”,然后点“Save”保存文件。这里文件将保存在桌面。

RStudio界面介绍

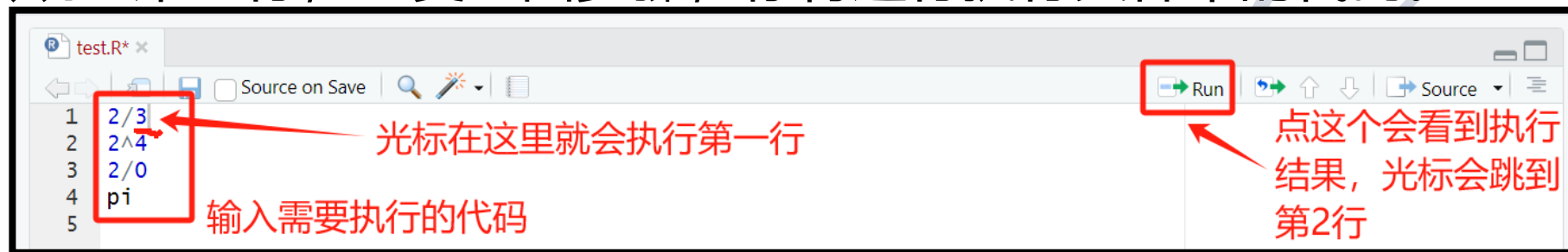
- Console: (左下角2区) 这里是执行代码的地方, 执行结果也会显示在这里。你可在这个区域直接输入代码, 然后按回车键输出结果。比如:

```
> 2/3  
[1] 0.6666667  
> 2^4  
[1] 16  
> 2/0  
[1] Inf  
> pi  
[1] 3.141593
```



RStudio界面介绍

- 你也可通过Source区执行代码。以刚才保存下的test.R文件来举例：将光标放在第一行，点Run键将执行这行代码，然后光标会自动会跳至第二行；重复上面步骤，你将逐行执行文件中的代码。



注意：光标不要求在行前或行末，可放在一行中的任意位置。

如果你想执行整个文件中的代码，可以点Source键执行，执行时点Source右边的小箭头，选Source with echo就可以在Console中看到所有行的执行结果。



RStudio界面介绍

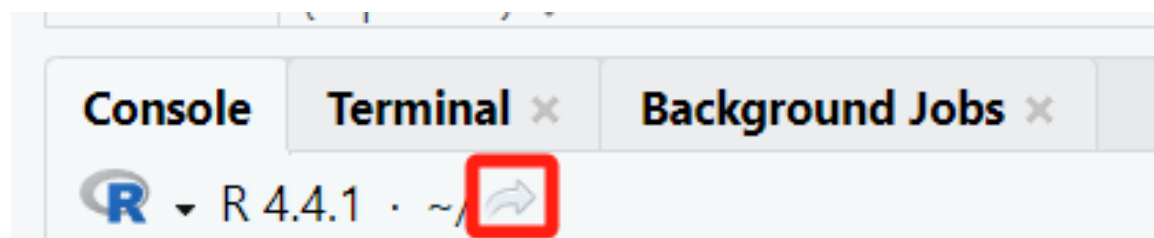
(3) Environment, History, Connections: (右上角3区)

- Environment: 是用来记录当前变量的数值, 方便查看当前变量的状况。
- History: 是所有在 Console区内代码执行的历史记录。
- Connections: 是用来方便连接外部数据库, 本课程不涉及, 先不介绍。

RStudio界面介绍

(4) Files, Plots, Packages, Help, Viewer: (右下角4区)

Files: 这里会显示当前工作路径下的文件，让使用者了解所在的工作路径，这对读写文件非常重要。如下图，当前的工作路径是默认工作路径，也显示了路径下所有文件。你可通过在Console输入`getwd()`函数来获取当前工作路径。如果Files下显示的不是当前工作路径，那是由于没及时更新视图所导致的。可点击Console字符串旁边箭头的来更新，如下图。其余窗口介绍参考实验讲义。



基本语法

赋值：

- 表示给某个变量赋予具体的值
- 使用=或<-

```
Console Terminal x Background Jobs x
R 4.4.1 · ~/
> x<-3
> y=4
> z=y+x
> print(x)
[1] 3
> print(y)
[1] 4
> print(z)
[1] 7
> a<-'xyz'
> print(a)
[1] "xyz"
> |
```



基本语法

运算：

- 有具体取值的多个变量可以进行其取值类型所允许的各类运算。
- 包括数值运算、比较运算、逻辑运算等。



```
Console Terminal x Jobs x
~/
> -9^3*5/6
[1] -607.5
> sqrt(3)
[1] 1.732051
> log(10)
[1] 2.302585
> log10(3+2*pi)
[1] 0.967697
> exp(3.2)
[1] 24.53253
> cos(2*pi)
[1] 1
> prod(7:4)/prod(3:1)
[1] 140
> prod(6:5)/prod(2:1)
[1] 15
> 1/prod(6:5)/prod(2:1)
[1] 0.01666667
> |
```

基本语法

向量：

- 一系列相同类型的数据的集合，可以是一组数值、一组逻辑值、一组字符串等

数值向量：

1. 使用函数 `c()` 可以实现将多个对象合并到一起：

```
c(1, 2, 3, 4, 5)
```

```
#> [1] 1 2 3 4 5
```

```
c(1, 2, c(3, 4, 5))
```

```
# 将多个数值向量合并成一个数值向量
```

```
#> [1] 1 2 3 4 5
```



基本语法

向量：

- 一系列相同类型的数据的集合，可以是一组数值、一组逻辑值、一组字符串等

数值向量：

2. 使用英文冒号或函数seq()可以创建等差的数值向量：

1:5

同 seq(5) 或 seq(1,5)

#> [1] 1 2 3 4 5

seq(1, 10, 2)

从 1 开始，到 10 结束，相邻数字间的差为 2

#> [1] 1 3 5 7 9

seq(3, length.out = 10)

从 3 开始，相邻数字间的差为1，一共有10个数

#> [1] 3 4 5 6 7 8 9 10 11 12



基本语法

向量：

- 一系列相同类型的数据的集合，可以是一组数值、一组逻辑值、一组字符串等

数值向量：

3. 使用rep()函数可以创建重复向量：

1:5

x = 1:3

从1开始，相邻数字间的差为1，到3介绍

rep(x, 2)

向量x重复2次

#> [1] 1 2 3 1 2 3

rep(x, each = 2)

向量x中的每个数重复2次

#> [1] 1 1 2 2 3 3



基本语法

向量：

- 一系列相同类型的数据的集合，可以是一组数值、一组逻辑值、一组字符串等

数值向量：

3. 使用rep()函数可以创建重复向量：

```
rep(x, c(2, 1, 2))
```

按照规则重复序列中的各元素，这里的规则表示第1个元素重复2次，第2个元素重复1次，第3个元素重复2次

```
#> [1] 1 1 2 3 3
```

```
rep(x, each = 2, length.out = 4)
```

向量x中的每个数重复2次，取其中前4个数

```
#> [1] 1 1 2 2
```

```
rep(x, times = 3, each = 2)
```

向量x重复3次，其中的每个数重复2次，

```
#> [1] 1 1 2 2 3 3 1 1 2 2 3 3 1 1 2 2 3 3
```



基本语法

向量：

- 一系列相同类型的数据的集合，可以是一组数值、一组逻辑值、一组字符串等

数值向量：

4. 向量可以做 +、-、*、/四则运算，即对应元素分别做运算的向量化运算。注意，R 中两个不同长度的向量做运算，短的会自动循环补齐以配合长的：

```
2:3 + 1:5
```

```
# (2, 3, 2, 3, 2) + (1, 2, 3, 4, 5)
```

```
#> [1] 3 5 5 7 7
```



基本语法

向量：

- 一系列相同类型的数据的集合，可以是一组数值、一组逻辑值、一组字符串等

逻辑向量：是由逻辑值（TRUE 或 FALSE, 或简写为 T 或 F）组成的向量。

1. 向量做逻辑运算，得到的结果是逻辑向量：

```
c(1, 2) > c(2, 1)
```

```
# 等价于 c(1 > 2, 2 > 1)
```

```
#> [1] FALSE
```

```
TRUE
```

```
c(2, 3) > c(1, 2, -1, 3)
```

```
# 等价于 c(2 > 1, 3 > 2, 2 > -1, 3 > 3)
```

```
#> [1] TRUE TRUE TRUE FALSE
```



基本语法

向量：

- 一系列相同类型的数据的集合，可以是一组数值、一组逻辑值、一组字符串等

逻辑向量：是由逻辑值（TRUE 或 FALSE, 或简写为 T 或 F）组成的向量。

2.元素属于运算符`%in%`：判断元素是否属于集合

```
c(1, 4) %in% c(1, 2, 3)
```

```
# 左边向量每一个元素是否属于右边集合
```

```
#> [1] TRUE FALSE
```



基本语法

向量：

- 一系列相同类型的数据的集合，可以是一组数值、一组逻辑值、一组字符串等

字符向量：字符（串）向量，是一组字符串组成的向量，单引号和双引号都可以用来生成字符串。

```
"hello, world!"
```

```
#> [1] "hello, world!"
```

```
c("Hello", "World")
```

```
#> [1] "Hello" "World"
```

```
c("Hello", "World") == "Hello, World"
```

```
#> [1] FALSE FALSE
```



基本语法

向量：

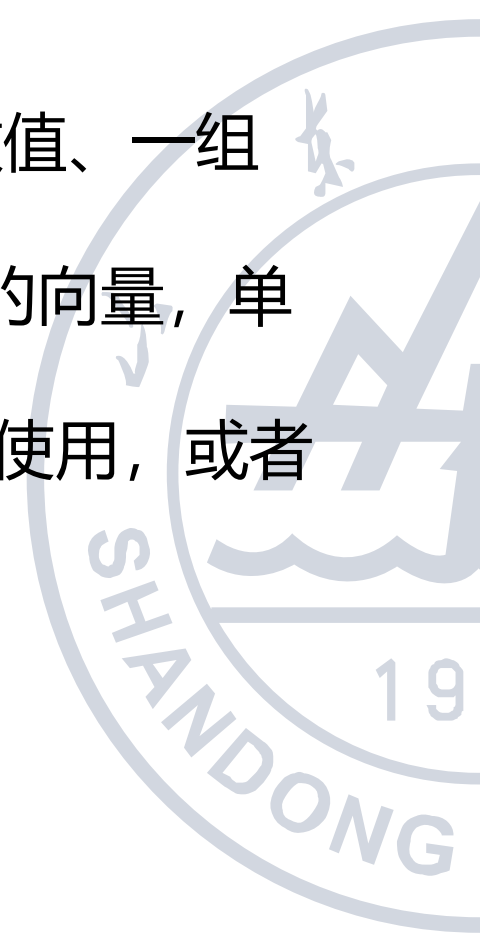
- 一系列相同类型的数据的集合，可以是一组数值、一组逻辑值、一组字符串等

字符向量：字符（串）向量，是一组字符串组成的向量，单引号和双引号都可以用来生成字符串。

- 若字符串本身包含单引号或双引号，可以错开使用，或者用转义符\做转义

```
'Is "You" a Chinese name?'
```

```
#> [1] "Is \"You\" a Chinese name?"
```



基本语法

向量：

- 一系列相同类型的数据的集合，可以是一组数值、一组逻辑值、一组字符串等

向量的访问：获取向量中某个位置的元素

- 使用元素的位置来访问：

```
v1 = c(1, 2, 3, 4)
```

```
v1[2]
```

```
# 第 2 个元素
```

```
v1[2:4]
```

```
# 第 2-4 个元素
```

```
v1[-3]
```

```
# 除了第 3 个之外的元素
```

```
v1[3:6]
```

```
# 可以访问不存在的位置，返回`NA`
```



基本语法

向量：

- 一系列相同类型的数据的集合，可以是一组数值、一组逻辑值、一组字符串等

向量的访问：获取向量中某个位置的元素

- 位置向量作为索引，注意不能既放正数又放负数：

```
v1[c(1, 3)]
```

```
v1[c(1, 2, -3)] # 报错
```



基本语法

向量：

- 一系列相同类型的数据的集合，可以是一组数值、一组逻辑值、一组字符串等

向量的访问：获取向量中某个位置的元素

- 使用逻辑向量来访问，输入与向量相同长度的逻辑向量，以此决定每一个元素是否要被获取，这可以引申为“根据条件访问向量子集”：

```
v1[c(TRUE, FALSE, TRUE, FALSE)]
```

```
v1[v1 <= 2]
```

```
# 同 v1[which(v1 <= 2)] 或 subset(v1, v1<=2)
```

```
v1[v1 ^ 2 - v1 >= 2]
```

```
which.max(v1)
```

```
# 返回向量 v1 中最大值所在的位置
```

```
which.min(v1)
```

```
# 返回向量 v1 中最小值所在的位置
```

基本语法

向量：

- 一系列相同类型的数据的集合，可以是一组数值、一组逻辑值、一组字符串等

向量的其他操作：

- 指定元素赋值：对向量子集赋值，就是先访问到向量子集，再赋值

```
v1[2] = 0
```

```
v1[2:4] = c(0, 1, 3)
```

```
v1[c(TRUE, FALSE, TRUE, FALSE)] = c(3, 2)
```

```
v1[v1 <= 2] = 0
```

```
v1[10] = 8
```

```
# 若对不存在的位置赋值，前面将用`NA`补齐v1
```

基本语法

向量：

- 一系列相同类型的数据的集合，可以是一组数值、一组逻辑值、一组字符串等

向量的其他操作：

- 元素排序
 - `sort()`: 对向量排序，默认 `decreasing = FALSE` 表示升序

```
x = c(1,5,8,2,9,7,4)
```

```
sort(x)
```

```
#> [1] 1 2 4 5 7 8 9
```

```
order(x)
```

```
# 默认升序，排名第 2 的元素在原向量的第 4 个位置
```

```
#> [1] 1 4 7 2 6 3 5
```

```
x[order(x)]
```

```
# 同 sort(x)
```



基本语法

向量：

- 一系列相同类型的数据的集合，可以是一组数值、一组逻辑值、一组字符串等

向量的其他操作：

- 元素排序
 - `order()`：返回元素排好序的索引，以其结果作为索引访问元素，正好是排好序的向量

order(x)

默认升序，排名第 2 的元素在原向量的第 4 个位置

```
#> [1] 1 4 7 2 6 3 5
```

x[order(x)]

同 `sort(x)`

```
#> [1] 1 2 4 5 7 8 9
```



基本语法

向量：

- 一系列相同类型的数据的集合，可以是一组数值、一组逻辑值、一组字符串等

向量的其他操作：

- 元素排序
 - `rank()`：返回该向量中各个元素的“排名”，参数 `method` 设置相同值的处理方法

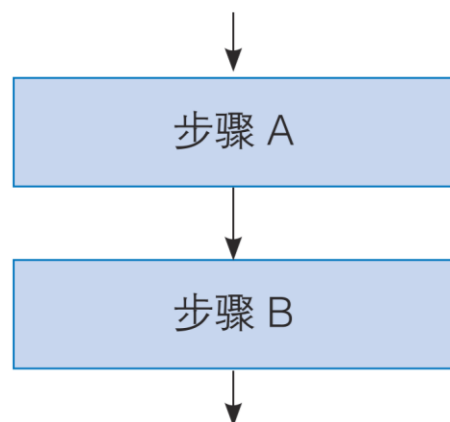
rank(x)

默认升序，第 2 个元素排名第 4 位

```
#> [1] 1 4 6 2 7 5 3
```



顺序结构



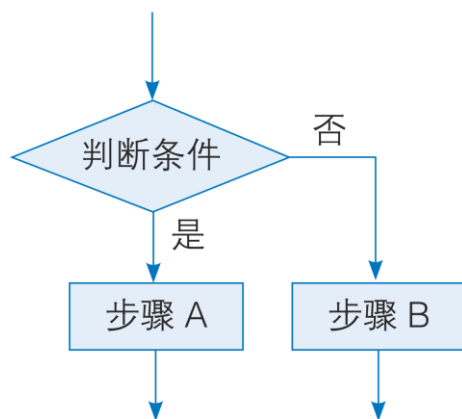
```
Console Terminal x Background Jobs x
R 4.4.1 ~/
> x<-3
> y=4
> z=y+x
> print(x)
[1] 3
> print(y)
[1] 4
> print(z)
[1] 7
> a<-'xyz'
> print(a)
[1] "xyz"
> |
```

A screenshot of an R console window. The window has three tabs: "Console", "Terminal", and "Background Jobs". The "Console" tab is active, showing the R prompt and the following code and output:

```
> x<-3
> y=4
> z=y+x
> print(x)
[1] 3
> print(y)
[1] 4
> print(z)
[1] 7
> a<-'xyz'
> print(a)
[1] "xyz"
> |
```



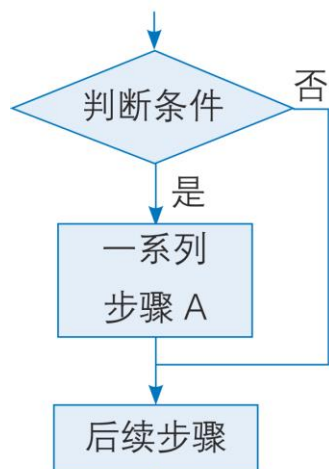
分支结构/选择结构



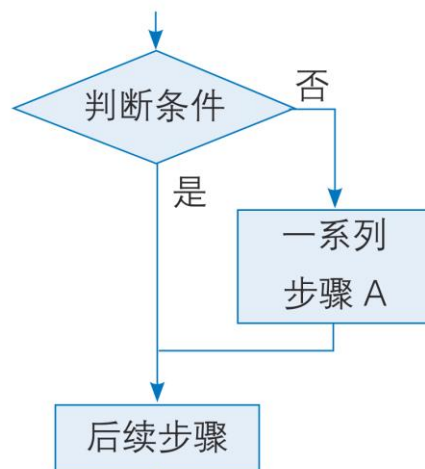
```
if(boolean_expression) {  
    // 如果布尔表达式为真将执行的语句  
} else {  
    // 如果布尔表达式为假将执行的语句  
}
```



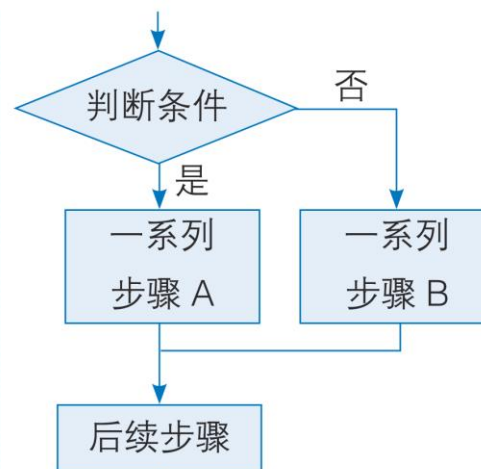
分支结构/选择结构



a. 单分支 1



b. 单分支 2



c. 双分支

```
if(boolean_expression) {  
    // 布尔表达式为真将执行的语句  
}  
if(!boolean_expression) {  
    // 布尔表达式为假将执行的语句  
}
```



分支结构/选择结构

多分支

`switch(expression, case1, case2, case3....)`

switch 语句必须遵循下面的规则：

- **switch** 语句中的 **expression** 是一个常量表达式，可以是整数或字符串，如果是整数则返回对应的case位置值，如果整数不在位置的范围内则返回 NULL。
- 如果匹配到多个值则返回第一个。
- **expression**如果是字符串，则对应的是 case 中的变量名对应的值，没有匹配则没有返回值。
- switch 没有默认参数可用。

分支结构/选择结构

代码示例

```
x <- 50L
if(is.integer(x)) {
  print("x 是一个整数")
}
```

```
x <- c("google", "runoob", "taobao")

if("runoob" %in% x) {
  print("包含 runoob")
} else {
  print("不包含 runoob")
}
```



分支结构/选择结构

代码示例

```
x <- c("google","runoob","taobao")
if("weibo" %in% x) {
  print("第一个 if 包含 weibo")
} else if ("runoob" %in% x) {
  print("第二个 if 包含 runoob")
} else {
  print("没有找到")
}
```



分支结构/选择结构

代码示例

```
x <- switch(  
  3,  
  "google",  
  "runoob",  
  "taobao",  
  "weibo"  
)  
print(x)
```



分支结构/选择结构

代码示例

```
you.like<-"runoob"  
switch(you.like, google="www.google.com",  
runoob = "www.runoob.com", taobao =  
"www.taobao.com")
```

```
> x <- switch(4, "google", "runoob", "taobao")  
> x  
NULL  
> x <- switch(4, "google", "runoob", "taobao")  
> x  
NULL
```

循环结构

计数循环

`for`(循环变量 `in` 序列) 循环语句

```
v <- LETTERS[1:4]
for ( i in v ) {
  print(i)
}
```

执行以上代码，输出结果为：

```
[1] "A"
[1] "B"
[1] "C"
[1] "D"
```



循环结构

while循环

```
v <- c("Google", "Runoob")  
cnt <- 2  
while (cnt < 7) {  
  print(v)  
  cnt = cnt + 1  
}
```



循环结构

repeat循环

```
repeat{
```

```
...
```

```
  if(循环退出条件) break
```

```
}
```

```
v <- c("Google", "Runoob")
```

```
cnt <- 2
```

```
repeat {
```

```
  print(v)
```

```
  cnt <- cnt+1
```

```
  if(cnt > 5) {
```

```
    break
```

```
  }
```

```
}
```



循环结构

循环控制

1. 用break语句可以退出所在的循环。

以下实例在变量 cnt 为 5 时使用 break 退出循环，cnt 为计数变量：

```
v <- c("Google", "Runoob")
cnt <- 2
repeat {
  print(v)
  cnt <- cnt+1
  if(cnt > 5) {
    break
  }
}
```



循环结构

循环控制

2. 用next语句可以进入所在循环的下一轮。

以下实例输出 26 个字母的前面 6 个字母，在字母为 D 的时候跳过当前的循环，进行下一次循环：

```
v <- LETTERS[1:6]
for ( i in v ) {
  if ( i == "D" ) {# D 不会输出，跳过这次循环，进入下
    next
  }
  print(i)
}
```

一次

什么是函数

函数是面向对象程序设计中，对功能的抽象，是一组一起执行一个任务的语句。

函数定义通常由以下几个部分组成：

- 函数名: 为函数指定一个唯一的名称，以便在调用时使用。
- 参数: 定义函数接受的输入值。参数是可选的，可以有多个。
- 函数体: 包含实际执行的代码块，用大括号 `{}` 括起来。
- 返回值: 指定函数的输出结果，使用关键字 `return`。

什么是函数

R语言中的函数定义使用function关键字，一般形式如下：

```
function_name <- function(arg_1, arg_2, ...)  
{  
  # 函数体  
  # 执行的代码块  
  return(output)  
}
```

说明：

- function_name : 为函数名
- arg_1, arg_2, ... : 形式参数列表



什么是函数

例：# 定义一个加法函数

```
add_numbers <- function(x, y) {  
  result <- x + y  
  return(result)  
}
```

调用函数

```
sum_result <- add_numbers(3, 4)  
print(sum_result) # 输出 7
```

我们定义了一个名为 **add_numbers** 的函数，它接受两个参数 **x** 和 **y**。函数体中的代码将这两个参数相加，并将结果存储在变量 **result** 中。最后，使用 **return** 关键字返回结果。

要调用函数，我们使用函数名后跟参数列表的形式，将参数的值传递给函数。在本例中，我们调用 **add_numbers** 函数，并传递参数 3 和 4。函数执行后，返回结果 7，我们将其存储在变量 **sum_result** 中，并打印输出。

自定义函数

在代码的前面通过函数定义的方式指定函数名、函数参数、函数内容和函数返回值，在代码中即可通过函数名(函数参数)的方式调用函数。

例：

```
new.function <- function(a) {  
  for(i in 1:a) {  
    b <- i^2  
    print(b)  
  }  
}  
  
# 调用函数，并传递参数  
new.function(6)
```



自定义函数

我们也可以创建一个不带参数的函数

例：

```
new.function <- function() {  
  for(i in 1:5) {  
    print(i^2)  
  }  
}
```

调用函数，不需要传递参数

```
new.function()
```



自定义函数的返回值

函数内容中有return的情况，调用函数会获取返回值；没有return的情况，函数将默认返回最后一条语句的计算结果。

例1:

```
sum_function <- function(a, b) {  
  result <- a + b  
  return(result) # 显式返回结果  
}  
print(sum_function(3, 5)) # 输出: 8
```

例2:

```
square_function <- function(x) {  
  x^2 # 返回最后的计算结果  
}  
print(square_function(4)) # 输出: 16
```



自定义函数的返回值

return()可以在复杂逻辑中提前退出函数，避免多余计算。例如：

```
is_positive <- function(x) {  
  if (x <= 0) {  
    return(FALSE) # 提前返回  
  }  
  return(TRUE)  
}  
print(is_positive(5)) # 输出: TRUE  
print(is_positive(-3)) # 输出: FALSE
```



自定义函数的返回值

函数可以返回列表或其他复杂对象，例如多个值：

```
multi_return <- function(a, b) {  
  list(sum = a + b, product = a * b)  
}  
result <- multi_return(3, 5)  
print(result$sum) # 输出: 8  
print(result$product) # 输出: 15
```

通过合理使用return()，可以提升代码的清晰度和可维护性。



带有参数值的函数

函数参数，可以按函数创建时的顺序来传递，也可以不按顺序，但需要指定参数名：

创建函数

```
new.function <- function(a,b,c) {  
  result <- a * b + c  
  print(result)  
}
```

不带参数名

```
new.function(5,3,11)
```

带参数名

```
new.function(a = 11, b = 5, c = 3)
```



带有参数值的函数

函数创建时也可以为参数指定默认值，如果调用的时候不传递参数就会使用默认值：

创建带默认参数的函数

```
new.function <- function(a = 3, b = 6) {  
  result <- a * b  
  print(result)  
}
```

调用函数，但不传递参数，会使用默认的
new.function()

调用函数，传递参数
new.function(9,5)



内置函数

R 语言提供了很多有用的内置函数，我们无需定义它就可以直接使用。例如：seq(), mean(), max(), sum(x) 以及 paste(...) 等。

输出 32 到 44 到的所有数字

```
print(seq(32,44))
```

计算两个数的平均数

```
print(mean(25:82))
```

计算 41 到 68 所有数字之和

```
print(sum(41:68))
```

可以在控制台通过?function_name或help("function_name")来查看内置函数的说明，例如：

```
> help("sum")
```

```
> ?mean
```



参考文献

- 张敬信(2022). *R 语言编程：基于 tidyverse*. 人民邮电出版社, 北京.
- 王敏杰(2021). 数据科学中的 *R* 语言. bookdown.org.
- 谢益辉(2021). *rmarkdown: Dynamic Documents for R*.
- 黄湘云(2021). *Github: R-Markdown-Template*.

